

Sustainable Software in Practice: A Use-Case-Oriented Tool for Exploring Sustainability Criteria

Anas Abukhadijah¹, Lejla Begic-Fazlic ¹, Achim Guldner ¹, Sebastian Weber ¹,
Sebastian Bast ¹, Guido Dartmann ¹, and Stefan Naumann ¹

Abstract: With digital transformation and the rise of artificial intelligence (AI), AI-based software systems have become essential to modern society, raising concerns about their environmental impacts. Sustainable software engineering integrates ecological considerations into software development processes. Existing sustainability frameworks and tools often lack interactivity and practical integration. This paper presents an interactive tool that introduces capabilities to support the accessibility and usability of a sustainability criteria catalog through dynamic filtering, interactive visual dashboards, and contextual markdown documentation. This approach supports decision-making, encourages practical application, and fosters feedback loops to refine sustainability standards.

Keywords: Sustainable software engineering, Software sustainability, Interactive tooling, Sustainability criteria, Green software, Decision support, Visual dashboards

1 Introduction

Software systems have increasingly become a vital part of modern infrastructure, significantly influencing economic growth, social interaction, and daily life [Ag23; KD11]. With the global acceleration of digital transformation and the rise of AI, the environmental impact associated with software systems - particularly their consumption of energy and resources - has gained significant attention in both academia and industry [La15; Na11; VSC23]. The intersection of software engineering and sustainability is addressed by the discipline of sustainable software engineering, which aims to integrate sustainability considerations into the software development life cycle to reduce environmental footprints while maintaining performance and functionality [Be15; Na11; Ve18].

Traditionally, software engineering prioritized functional correctness, performance optimization, and maintainability. However, increasing awareness of climate change and environmental degradation requires extending these priorities to include ecological sustainability [CP17]. Researchers and practitioners now recognize that software indirectly influences environmental sustainability through its operational resource demands, hardware utilization, and life-cycle management [Hi16]. Consequently, a systematic and rigorous approach to evaluating, managing, and reducing software-related environmental impacts has become essential.

¹ University of Applied Sciences Trier, Environmental Campus Birkenfeld, Institute for Software Systems, PO-Box 1380, 55761 Birkenfeld, Germany, a.abukhadijah@umwelt-campus.de; l.begic@umwelt-campus.de; a.guldner@umwelt-campus.de; Seb.Weber@umwelt-campus.de; s.bast@umwelt-campus.de; g.dartmann@umwelt-campus.de; s.naumann@umwelt-campus.de

Efforts such as the GREENSOFT reference model by Naumann et al. [Na11], the Green Software Measurement Model (GSMM) by Guldner et al. [Gu24], and the Sustainability Assessment Framework (SAF) Toolkit by Lago et al. [La25] offer foundational knowledge and practical frameworks for evaluating and improving software sustainability. Despite these advancements, many existing models and tools face limitations in terms of interactivity and contextual applicability, particularly regarding their integration into real-world development processes and decision-making workflows. For instance, GREENSOFT Naumann et al. [Na11] and GSMM Guldner et al. [Gu24] provide structured models and metrics but lack contextual guidance and do not support collaborative annotation. Users in our internal projects KIRA², EASY³, Green Twin⁴, as well as partners from industry and other sectors, have often pointed out that the large number of criteria, indicators, and metrics leads to cluttered presentations. This makes it difficult for them to quickly identify and apply the most relevant information in practice. These observations served as the motivation for developing a tool specifically designed to address this issue.

To address this gap, we introduce a tool that provides an interactive interface for navigating sustainability criteria catalogs introduced e.g. from Kern et al. [Ke18], or measurement procedures from Guldner et al. [Gu24]. Users can apply predefined or dynamic filters to explore relevant indicators based on project goals, domain contexts, or constraints. Furthermore, the tool includes a Markdown-based Wiki system for documenting use-cases and linking each to selected criteria along with their explanation. This approach supports transparency, reproducibility, and knowledge sharing in sustainability-oriented software development. The purpose of this project is to make the sustainability criteria catalog more accessible to companies, decision makers, and developers, potentially leading to valuable feedback for the scientific community and informing future research.

2 Related Work

Research in sustainable software engineering encompasses diverse tools, frameworks, and educational initiatives aimed at enhancing software sustainability through energy-efficient practices and life-cycle considerations.

Naumann et al. [Na11] introduced conceptual models and practical guidelines promoting green software development, emphasizing energy efficiency and life-cycle-oriented design. Their work significantly influenced eco-labeling standards such as the Blue Angel for Resources and Energy-Efficient Software Products (DE-UZ 215)⁵, which stipulates strict criteria regarding energy and hardware usage, as well as transparency and update criteria.

² <https://www.umwelt-campus.de/iss/projekte/laufende-projekte/kira> [2025-06-18]

³ <https://www.umwelt-campus.de/iss/projekte/laufende-projekte/easy> [2025-06-18]

⁴ <https://www.umwelt-campus.de/iss/projekte/laufende-projekte/greentwin> [2025-06-18]

⁵ <https://www.blauer-engel.de/en/productworld/software> [2025-06-11]

Other foundational concepts or frameworks such as the Sustainability Assessment Framework (SAF) Toolkit [La25] have been developed. SAF specifically enables software architects to explicitly model sustainability as a core quality attribute during the software design phase. However, it primarily addresses design-time considerations and does not account for runtime interactions or project-specific customization. Moreover, its usage is often limited to expert-driven modeling workflows and lacks interactive support for practitioners during real-world decision-making processes. In contrast, our tool bridges this gap by offering a lightweight, browser-based interface that integrates markdown-based contextualization, dynamic filtering, and visual dashboards—allowing both technical and non-technical users to explore, apply, and document sustainability criteria in concrete project settings. To complement these design-focused frameworks, various runtime measurement tools have emerged to provide quantitative insights into software sustainability during actual operation. CodeCarbon [Co24], for instance, estimates the carbon footprint of Python code execution by accounting for the execution environment and geographical context. Furthermore, it does not support contextual linking to project-specific sustainability goals or provide interactive visual analytics beyond post-hoc dashboards, which limits its utility in integrated decision-making environments. Similarly, the Green Metrics Tool [Gr24a] provides runtime energy profiling capabilities for applications, assisting developers in identifying and addressing energy-inefficient code segments. This tool emphasizes container-based runtime profiling but does not natively support linking energy measurements to contextual use-case descriptions, nor does it allow practitioners to document rationale or reference criteria directly within the tool. Additionally, GreenHub SDK [Gr24b] specifically targets energy monitoring for Android applications. Its focus is primarily on data acquisition and model training rather than on providing practitioners with interactive, project-specific tools.

Addressing sustainability in the context of AI, the GAISSA project [MFD23] emphasizes integrating sustainability into AI-based software system design. GAISSA offers architecture-centric methods and tools that assist data scientists and software engineers in creating energy-efficient AI applications, introducing the novel concept of "greenability" to evaluate and optimize AI systems' sustainability through informed architectural decisions.

Educational initiatives, such as the Sustainable Software Engineering course [De24], attempt to bridge theoretical concepts with practical applications. However, existing educational resources and research tools often lack interactive, use-case-specific exploration capabilities and systematic documentation frameworks, which are essential for effectively justifying and contextualizing sustainability decisions within practical projects.

Guldner et al. [Gu24] proposed the Green Software Measurement Model (GSMM), a comprehensive reference framework designed to structure the measurement and analysis of the energy and resource consumption of software. The GSMM synthesizes insights from various research groups into a structured model comprising five key components: measured objects and goals, metrics, procedural models, measurement setups, and data evaluation protocols. This model emphasizes detailed metadata collection, enhancing reproducibility, comparability, and extensibility of sustainability assessments. Complementing the GSMM,

the structured criteria catalog introduced in Kern et al. [Ke18] (and updated in the process of the creation of the Blue Angel) offers a detailed collection of sustainability criteria derived from empirical software evaluations. Building on this foundation, the KIRA ⁶ project has extended the criteria catalog to include additional AI-specific sustainability aspects, reflecting recent developments and domain-specific challenges. These extended criteria are already applied in practical scenarios such as the determining the fill level of containers Cetkin et al. [Ce24]. Despite its comprehensiveness, the static nature of this catalog limits its usability due to the absence of interactive exploration features, dynamic filtering capabilities, and contextual documentation support.

To address these identified gaps, we developed the NEXUS Toolbox (Navigating Environmental Criteria (cross-criteria), Usage and Sustainability). This tool enhances existing sustainability criteria catalogs by providing an interactive environment for exploration, documentation, and visualization. It integrates embedded documentation features, enabling practitioners to effectively align their project-specific decisions with sustainability objectives through intuitive, context-aware interactions. In this way, the NEXUS Toolbox positions itself as a practice-oriented layer that contextualizes existing criteria and supports sustainability-related decision-making through concrete use-case guidance.

3 System Architecture

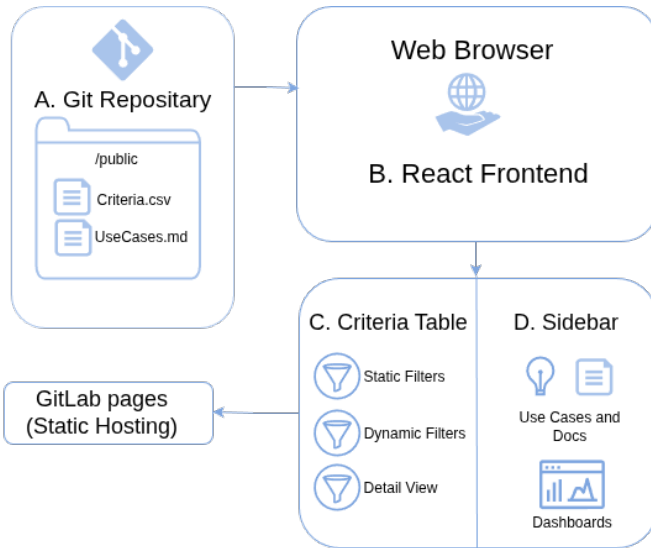


Fig. 1: System architecture

⁶ <https://www.umwelt-campus.de/iss/projekte/laufende-projekte/kira> [2025-06-18]

The tool's structure follows a modular layout (see Fig. 1). It is implemented as a React-based frontend and deployed via GitLab Pages (Fig. 1 B.). A central component is a dynamic table (Fig. 1 C.) that displays all available criteria and provides sorting and filtering functionalities. Users can use static filters in the form of drop-down menus that allow for rapid selection of criteria by predefined categories such as *type of indicator*, *effort*, *domain* or *use case*. Additionally, dynamic filters enable more flexible querying: users can freely select any column as a filter and enter search terms to refine the dataset according to their needs.

Furthermore, the system can display details for each individual criterion. When a table entry is selected, a separate view displays the available information associated with that criterion, including its definition, context of use, and measurement details (Fig. 1 C.) . This design facilitates focused inspection of single evaluation elements without distraction from the broader dataset.

In addition to information on the criteria, the toolbox integrates a sidebar interface (Fig. 1 D.) that lists practical use-cases. These are written in Markdown and rendered directly within the application using a client-side Markdown parser. Each use-case outlines a scenario in which certain criteria were applied, providing context, rationale, and relevant background information.

The architecture also includes support for embedded visual dashboards within the Markdown pages (Fig. 1 D., bottom). These dashboards allow users to interactively generate diagrams based on selected metrics and models. This functionality enhances the tool's analytical capabilities and helps users identify patterns and relationships in the data. Since the tool is deployed via GitLab Pages, there is no backend, database, or server-side logic involved (Fig. 1 A.). Leveraging this fully client-side architecture, the NEXUS Toolbox runs entirely in the browser and requires no installation or additional infrastructure—only a modern web browser is needed, which ensures ease of deployment and broad accessibility.

4 Implementation

The front-end was implemented in JavaScript⁷ using the React framework⁸. The dynamic criteria table (see Fig. 2) is powered by the open-source library `react-table`⁹, while Markdown rendering is handled via `react-markdown`¹⁰. All criteria data in CSV format and use-case documents in Markdown format are stored statically in the public directory of a GitLab repository. The application fetches this content at runtime using asynchronous requests.

⁷ ECMAScript 2021 standard (ES12).

⁸ React version 18.2.0, released June 2022.

⁹ <https://www.npmjs.com/package/react-table> [2025-06-13]

¹⁰ <https://github.com/remarkjs/react-markdown> [2025-06-13]

	Category Area	Criterion	Subcriterion	Indicator
1.1.1.1.a	Resource Efficiency	Hardware Efficiency	requirements and resulting hardware requirements (incl. peripheral devices)	Recommended local pr according to manufactu specifications
1.1.1.1.b	Resource Efficiency	Hardware Efficiency	Recommended system requirements and resulting hardware requirements (incl. peripheral devices)	Recommended local m according to manufactu specifications

Fig. 2: Table view of all criteria

To enable focused exploration of the criteria, the tool incorporates two filtering mechanisms. Static filters are implemented as drop-down menus that allow users to filter by predefined categories such as *indicator type*, *domain relevance*, *required effort* or *use case*. In addition, dynamic filters (see Fig. 3) allow users to interactively select any table column and enter free-text search queries to narrow down the displayed results based on their specific interests.

Capture Effort:

Medium

Indicator Type:

Qualitative

Category:

Resource Efficiency

Use Case:

Text Generation with LLMs

ID

1.1

Entfernen

Filter

Reset Filters

Add Filter

Advanced Options

Fig. 3: Static and dynamic Filters

Moving from general criteria exploration to specific practical contexts, the tool incorporates a detailed use-case documentation. To effectively visualize the practical application of sustainability criteria, interactive dashboards are embedded directly within each use-case page, using the recharts¹¹ library. Each use-case page clearly outlines a specific scenario,

¹¹ <https://recharts.org/en-US> [2025-06-13]

detailing the context, explanation, and relevance of selected criteria. The visual dashboards within these pages generate interactive bar and line charts, enabling users to examine and interpret metrics, evaluate algorithmic performance, and compare models in an accessible and engaging manner. As an exemplar, the waste management use case estimates the fill level of municipal waste bins. By fusing spatial impulse-response analysis with machine-learning techniques, this sound-based approach remains low-cost and non-invasive, and the same pipeline can be transferred to any application that requires reliable level or occupancy detection far beyond waste-management scenarios. This direct integration of visual analytics within use-cases enhances clarity, supports informed decision-making and provides deeper insight into the practical impacts of sustainability criteria application (see Fig. 4).

Interactive Visualization for the Use Case: Training on Local Hardware – SIREC (Waste Management)

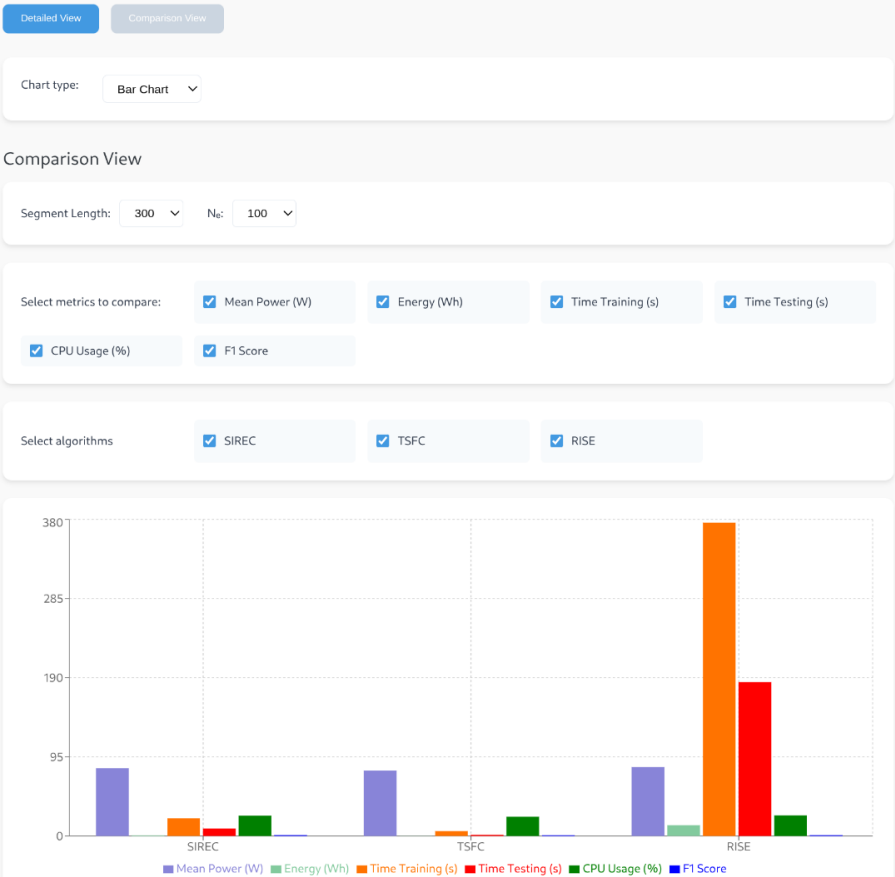


Fig. 4: Charts of a use-case

All filtering, navigation, and rendering are handled entirely within the browser. This approach ensures full transparency and version control through Git, and allows researchers and practitioners to easily extend or adapt the tool to their own needs. The tool is available as an open-source repository ¹², with a live instance ¹³.

5 Discussion and Outlook

This tool represents an attempt to bridge the gap between comprehensive sustainability frameworks and their practical application in software projects. By offering a structured and interactive interface, it enables developers and decision-makers to explore the extensive sustainability criteria catalog proposed by Naumann, Guldner, Kern et al. [Gu24; Ke18; NGK21], which may otherwise appear too complex or abstract for practical use.

Beyond simply listing metrics, the tool provides context and explanations for selecting specific criteria in defined use-cases. This feature is essential for supporting transparent decision-making, as it allows users to understand not only what to measure but also why certain criteria are relevant in their domain or project setting. In doing so, the tool facilitates deeper reflection on sustainability goals and their operational relevance.

The tool also opens a channel for feedback between practitioners and researchers. Developers and decision-makers can engage with the criteria in realistic settings, and their experiences—ranging from applicability to limitations—can be systematically documented and used to refine the criteria catalog itself. This creates a valuable feedback loop. While the current version of the tool demonstrates the concept, it remains a work in progress. Future development will focus on extending functionality, improving usability, and integrating more diverse use-cases and practitioner feedback.

To prevent pseudo-dependencies between criteria filters, we plan to implement a filter-validation module. Before executing a query, selected filters will be checked against predefined rules (e.g., certain domains only apply to specific indicator types). If an invalid combination is detected, the user will be alerted with suggestions for valid filter options.

For the next phase, we plan to conduct a user evaluation to ensure the practical applicability and usability of the tool. We will recruit 10 experts from industry and academia, perform semi-structured interviews, and collect feedback using the System Usability Scale. The collected results will then be analyzed to refine the NEXUS Toolbox based on practitioner suggestions.

Additionally, to further enhance the practical applicability and personalization of the tool, a promising future direction involves integrating a recommender system. This system would suggest relevant sustainability criteria based on project-specific metadata. By analyzing

¹² <https://gitlab.rlp.net/green-software-engineering/KIRA-Toolbox> [2025-06-20]

¹³ <https://kira-toolbox-4afdf7.pages.gitlab.rlp.net/> [2025-06-17]

characteristics such as project domain, development phase, target platform, team size, or known sustainability goals, the recommender system could intelligently prioritize and propose the most relevant indicators and metrics from the criteria catalog. This approach would support users in rapidly identifying context-appropriate sustainability considerations, reduce cognitive load, and facilitate adoption in time-constrained development environments. Recommendation algorithms could range from simple rule-based mappings to more sophisticated machine learning models trained on annotated use-case data and historical project feedback.

Acknowledgement

This project was funded by the Federal Ministry for the Environment, Climate Action, Nature Conservation and Nuclear Safety (BMUKN) project "KIRA" under Grant 67KI32013. Parts of the text have been enhanced and linguistically revised using AI tools.

References

- [Ag23] Agustian, K. et al.: The Impact of Digital Transformation on Business Models and Competitive Advantage. *Technology and Society Perspectives (TACIT)* 1 (2), pp. 79–93, 2023.
- [Be15] Becker, C. et al.: The Karlskrona Manifesto for Sustainability Design. In: *Proceedings of the 37th International Conference on Software Engineering - Volume 2*. IEEE, pp. 467–476, 2015.
- [Ce24] Cetkin, B. et al.: Spatial Impulse Response Analysis and Ensemble Learning for Efficient Precision Level Sensing. *Discover Artificial Intelligence* 4, p. 63, 2024.
- [Co24] CodeCarbon Contributors: CodeCarbon: Track the Carbon Footprint of Your Computing, Accessed: 2025-05-15, 2024, <https://codecarbon.io/>.
- [CP17] Calero, C.; Piattini, M.: *Green in Software Engineering*. Springer International Publishing, Cham, 2017.
- [De24] Delft University of Technology: Sustainable Software Engineering Course, Accessed: 2025-05-15, 2024, https://luiscruz.github.io/course_sustainableSE/2024.
- [Gr24a] Green Metrics Tool Contributors: Green Metrics Tool, Accessed: 2025-05-15, 2024, <https://greenlab.di.uminho.pt/green-metrics-tool>.
- [Gr24b] GreenHub Contributors: GreenHub: Energy-Aware Android App Analysis Platform, Accessed: 2025-05-15, 2024, <https://greenhub.ai/>.
- [Gu24] Guldner, A. et al.: Development and Evaluation of a Reference Measurement Model for Assessing the Resource and Energy Efficiency of Software Products and Components—Green Software Measurement Model (GSMM). *Future Generation Computer Systems* 155, p. 86, 2024.
- [Hi16] Hindle, A.: Green Software Engineering: The Curse of Methodology. In: *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. IEEE, pp. 46–55, 2016.

- [KD11] Kitchin, R.; Dodge, M.: *Software and Everyday Life*. MIT Press, Cambridge, MA, 2011.
- [Ke18] Kern, E. et al.: Sustainable software products—Towards assessment criteria for resource and energy efficiency. *Future Gener. Comput. Syst.* 86, pp. 199–210, 2018.
- [La15] Lago, P. et al.: Exploring Initial Challenges for Green Software Engineering: Summary of the First GREENS Workshop. *ACM SIGSOFT Software Engineering Notes* 40 (6), pp. 36–39, 2015.
- [La25] Lago, P. et al.: The Sustainability Assessment Framework Toolkit: A Decade of Modeling Experience. *Software and Systems Modeling* 24, Theme Section Paper, pp. 361–383, 2025.
- [MFD23] Martínez-Fernández, S.; Franch, X.; Durán, F.: Towards Green AI-Based Software Systems: An Architecture-Centric Approach (GAISSA). In: *2023 49th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. IEEE, Durrës, Albania, pp. 411–418, 2023.
- [Na11] Naumann, S. et al.: The GREENSOFT Model: A Reference Model for Green and Sustainable Software and its Engineering. *Sustainable Computing: Informatics and Systems* 1 (4), pp. 294–304, 2011.
- [NGK21] Naumann, S.; Guldner, A.; Kern, E.: The Eco-label Blue Angel for Software—Development and Components. In (Page, B. et al., eds.): *Advances and New Trends in Environmental Informatics*. Springer, Cham, pp. 55–67, 2021.
- [Ve18] Venters, C. C. et al.: Software Sustainability: Research and Practice from a Software Architecture Viewpoint. *Journal of Systems and Software* 138, pp. 174–188, 2018.
- [VSC23] Verdecchia, R.; Sallou, J.; Cruz, L.: A Systematic Review of Green AI. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 13 (4), e1507, 2023.